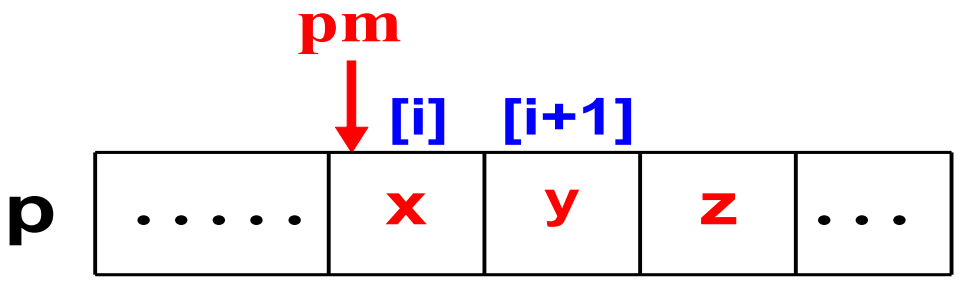
Memorarea elementelor unei multimi

Intr-un vector

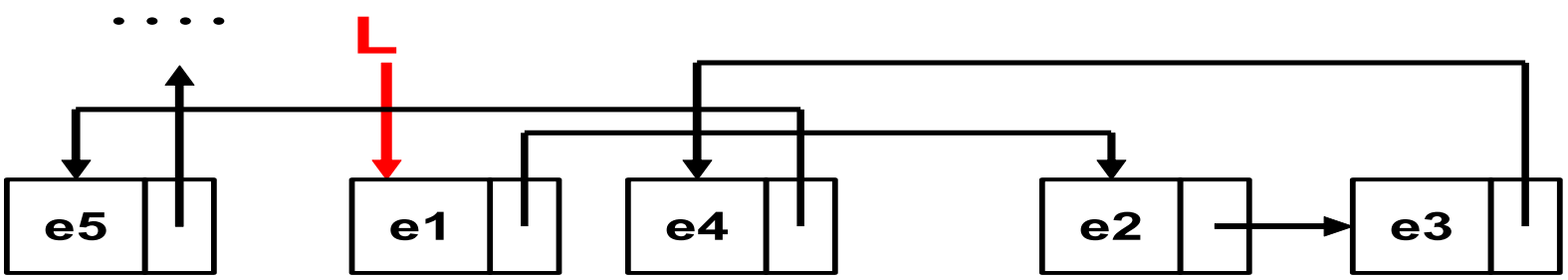
- elementele ocupa zone de memorie adiacente
- se aloca, static sau dinamic, spatiu pentru numarul maxim de elemente
- este posibila adresarea indexata
- inserarea / eliminarea unui element in / din interiorul colectiei implica deplasarea succesorilor la dreapta / stanga



Memorarea elementelor unei multimi

Intr-o lista, de exemplu simplu inlantuita

- elementele ocupa zone de memorie neadiacente

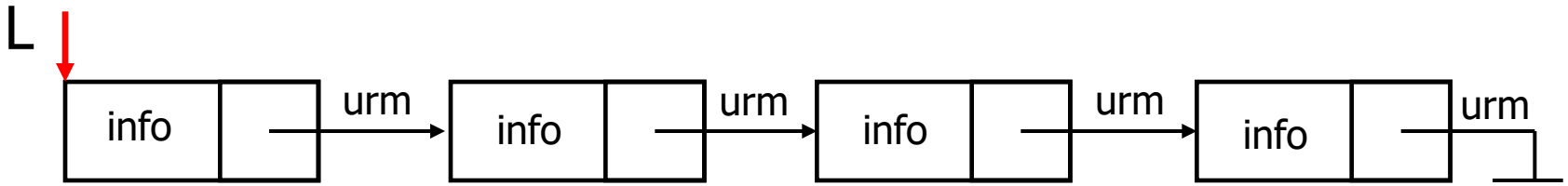


- fiecare element trebuie insotit de cel putin o informatie de legatura - adresa succesorului.
- Elementul si informatia de legatura se grupeaza intr-o structura numita celula.

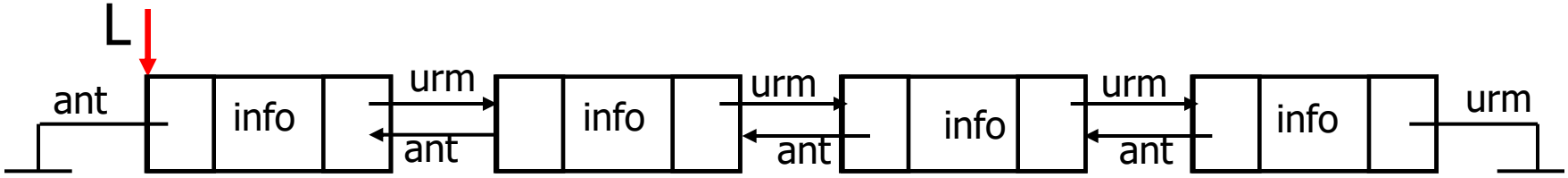
Liste

- O lista inlantuita este o colectie de elemente, alocate dinamic, dispersate in memorie, dar legate intre ele prin pointeri, ca intr-un lant

Lista simplu inlantuita



Lista dublu inlantuita

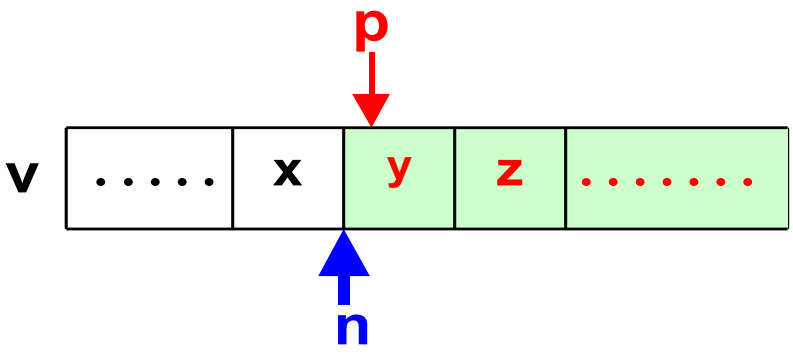


Liste

- o lista inlantuita este o structura dinamica, flexibila, care se poate extinde continuu (singura limita este dimensiunea zonei "heap" din care se aloca memorie)
- alocarea de spatiu se realizeaza la nivel de celula, numai atunci cand este necesar
- pentru localizarea unui element trebuie parcursa lista predecesorilor
- inserarea in lista / eliminarea din lista nu necesita deplasarea altor elemente, ci alocarea, respectiv eliberarea de spatiu si actualizarea unor informatii de legatura.

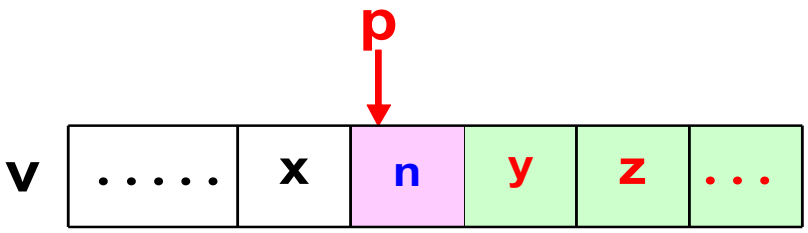
Inserare in lista

Inserarea in lista simplu inlantuita



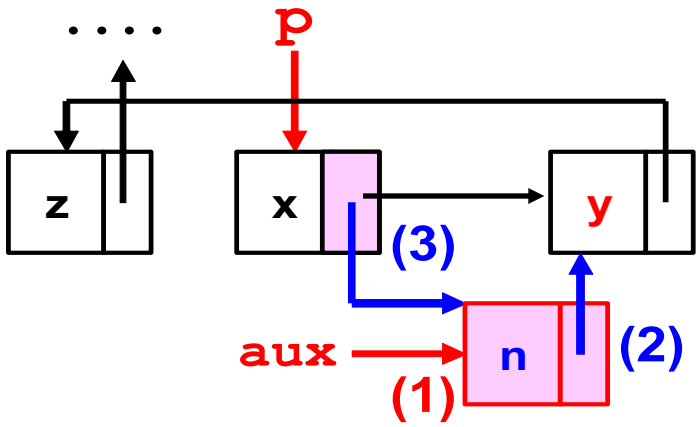
Vector:

- deplaseaza dreapta succesorii
- copiaza noul element



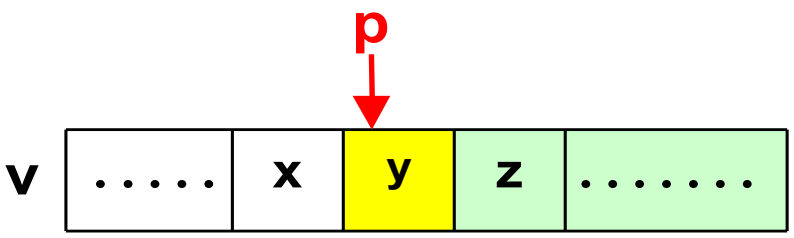
Lista:

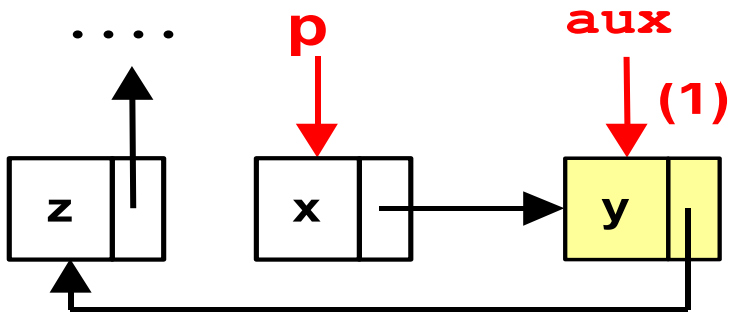
- aloca spatiu pentru o celula si copiaza noul element (1)
- actualizeaza legaturi (2,3)



Eliminare din lista

Eliminarea din lista simplu inlantuita



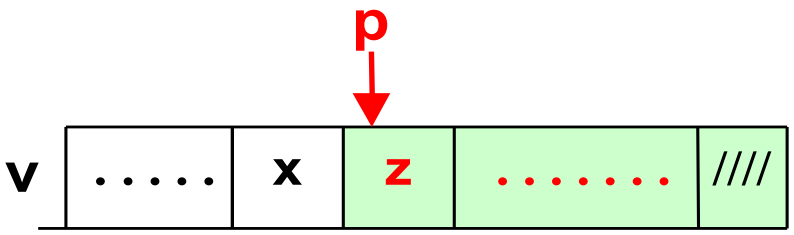


Vector:

- deplaseaza stanga succesorii

Lista:

- memoreaza adresa celulei eliminate (1)
- actualizeaza legatura (2)
- elibereaza spatiul ocupat de celula eliminata (3)



Operatii liste

Operatii cu elemente:

- Alocarea unei celule de lista
- Cautarea unui element in lista
- Inserare in lista
  - La inceputul listei
  - La sfarsitul listei
  - In interiorul listei: dupa / inaintea unui element specificat
- Eliminarea unui element din lista
- Modificare element
- Citirea unei liste
- Distrugere lista

Operatii liste

Operatii cu liste

- Iterare asupra listei (afisarea unei liste)
- Sortarea unei liste
- Inversarea unei liste
- Concatenarea a doua liste

Liste simplu inlantuite

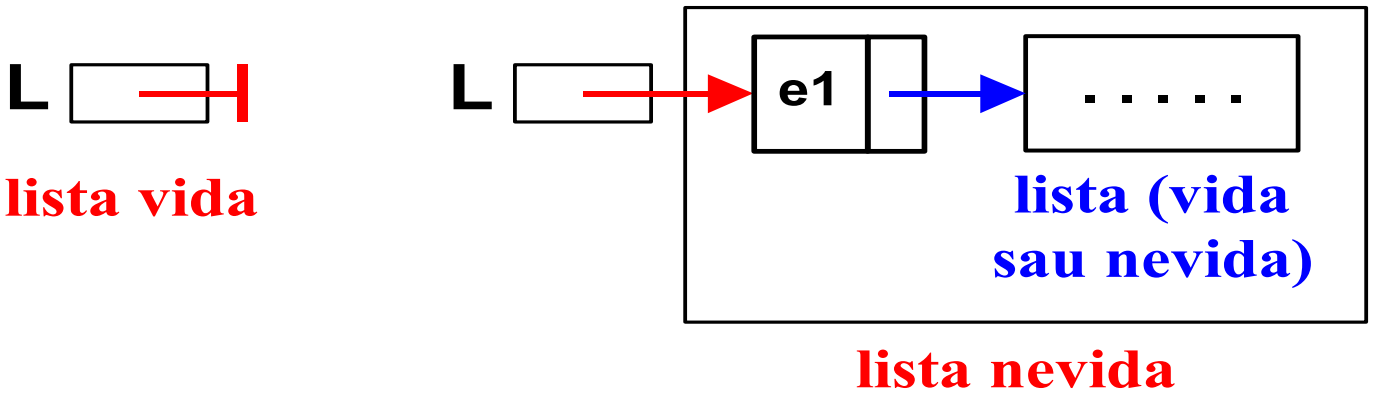
Lista este o structura recursiva, care se defineste:

```
lista: lista_vida | informatie lista
```

Informatia este specifica fiecarui element.

O variabila de tip lista are ca valoare

- **NULL** daca lista este vida
- **adresa primei celule** daca lista este nevida



Liste simplu inlantuite

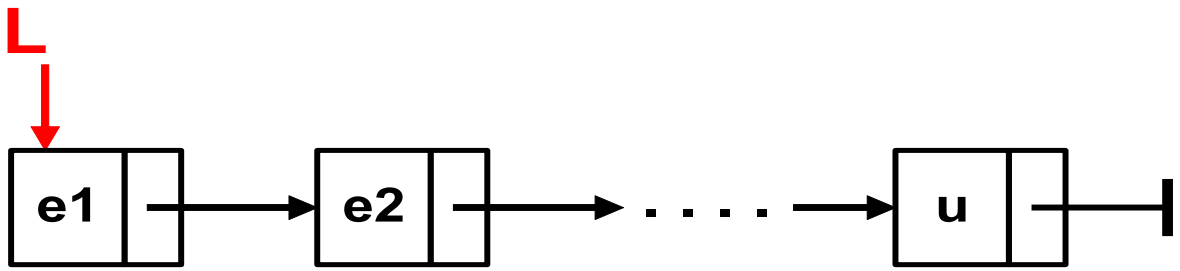
Se considera tipul elementelor unei liste: **TEL**.

Tipurile celula, lista si adresa lista se definesc astfel:

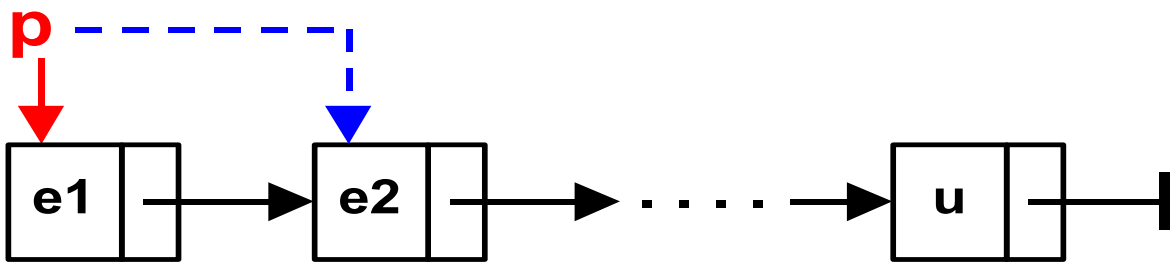
```
typedef int TEL;

typedef struct celula
  { TEL info;
    struct celula * urm;
  } TCelula, * TLista, ** ALista;

Tlista L;
```



Liste simplu inlantuite



Lista vida: `TLista p = NULL;`

test lista vida: `(p == NULL)`

valoarea primului element: `p->info`

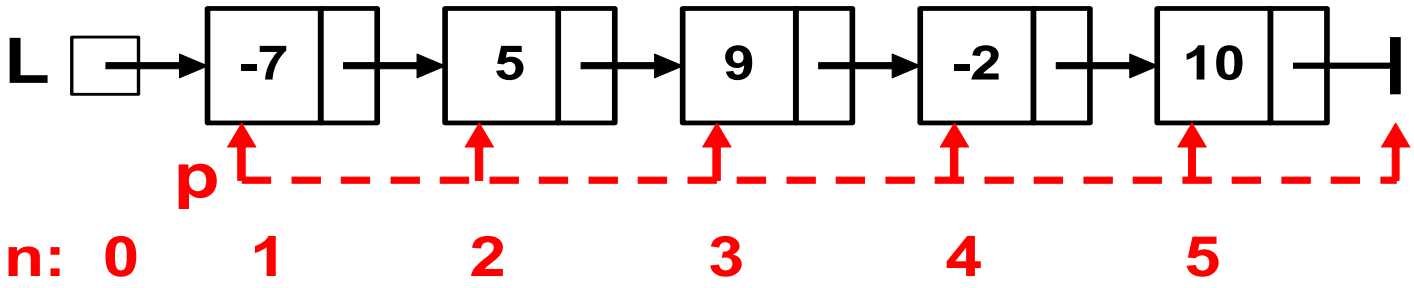
adresa urmatoarei celule: `p->urm`

valoarea urmatorului element: `(p->urm->info)`

avans la urmatoarea lista (celula): `p = p->urm`

Parcurgerea unei liste

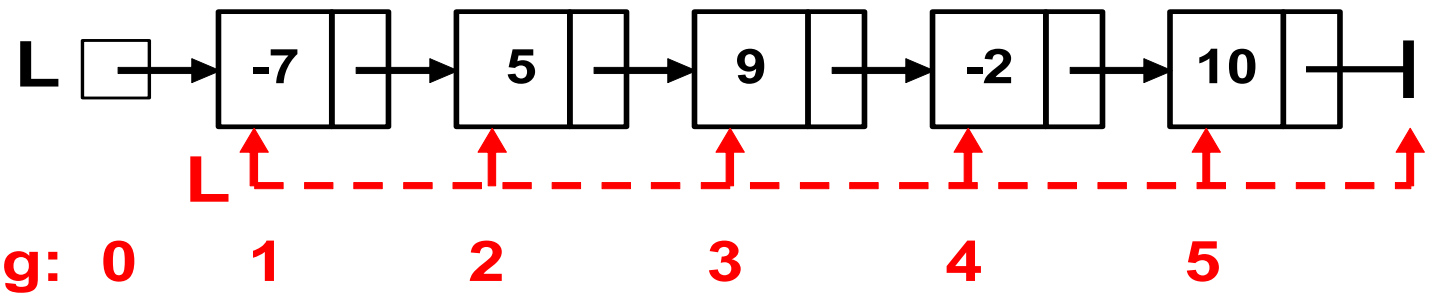
```
Parcurgere Lista
{ initializari;
  pentru fiecare celula din lista
  { prelucreaza info celula;}
}
```



Afisare elemente lista

```
void AfisareL(TLista L)
/* afiseaza valorile elementelor din lista */
{
  for (; L != NULL; L = L->urm)
    printf("%d ", L->info);
}
```

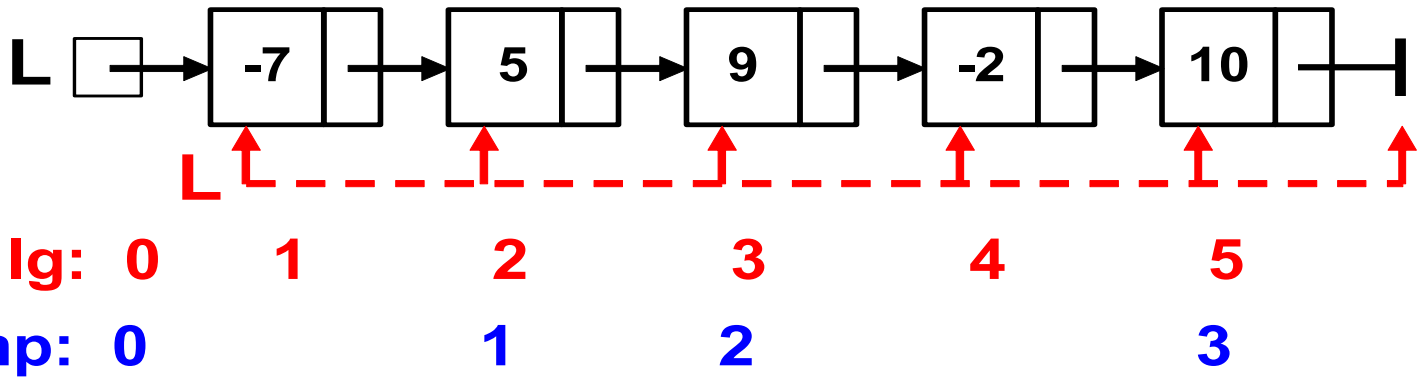
Parcurgere iterativa lista



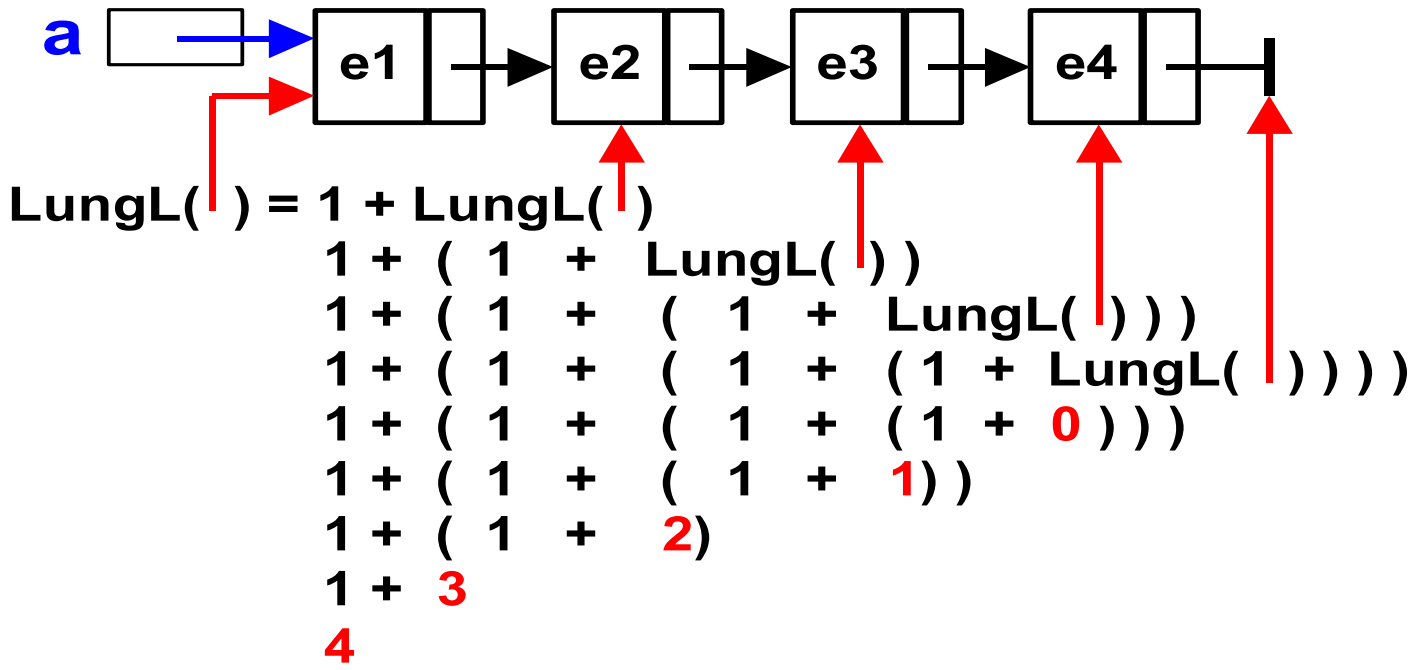

```
size_t LungimeI(TLista L)
/* varianta iterativa */
{ size_t lg = 0;
  for( ; L != NULL; L = L->urm)
     lg++;
  return lg;
}
```

Parcurgerea unei liste

```
Numar elemente pozitive
/* np = numarul de elemente pozitive */
{ initializari;
  pentru fiecare celula din lista
  {
     daca element > 0 atunci creste np;
  }
}
```

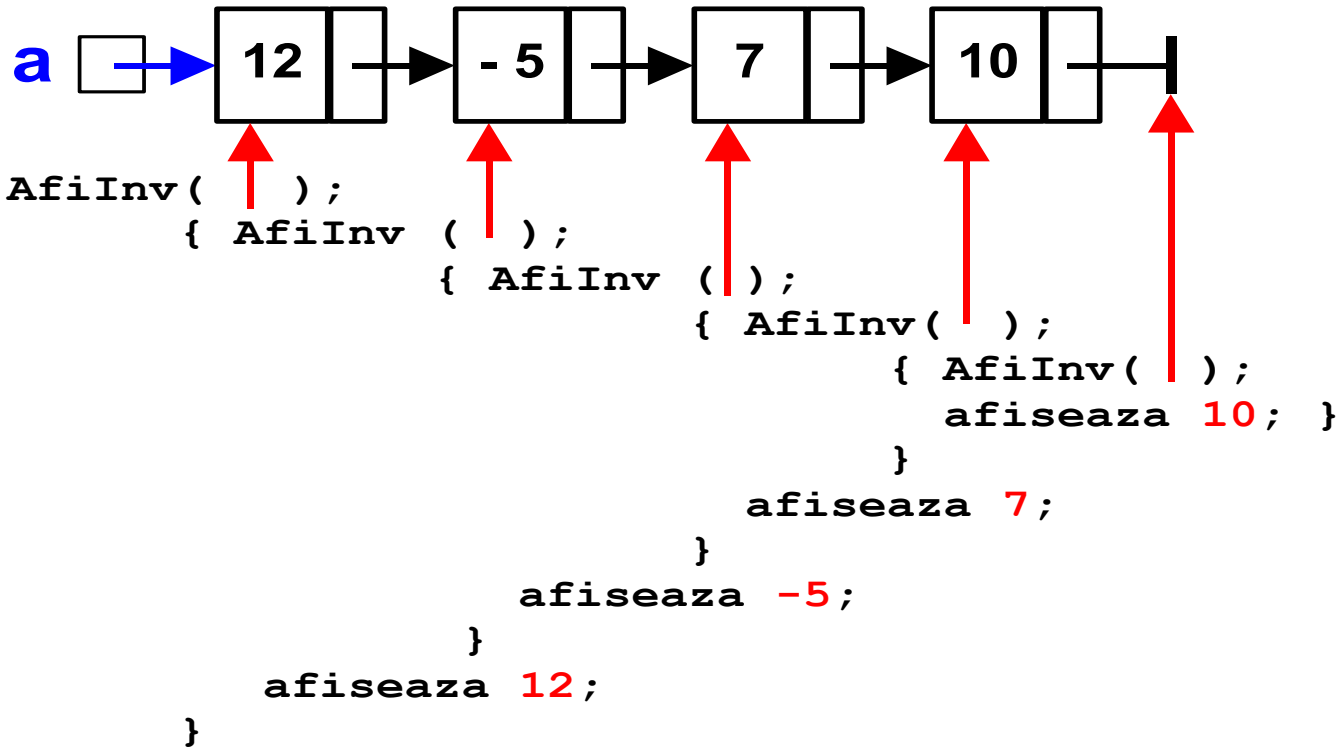


Parcurgerea recursiva a unei liste




```
Algoritm LungL(lista)
{ daca lista vida
     atunci intoarce 0;
     altfel intoarce 1 + LungL(listaUrmatoare);
}
```

Afisare continut lista in ordine inversa



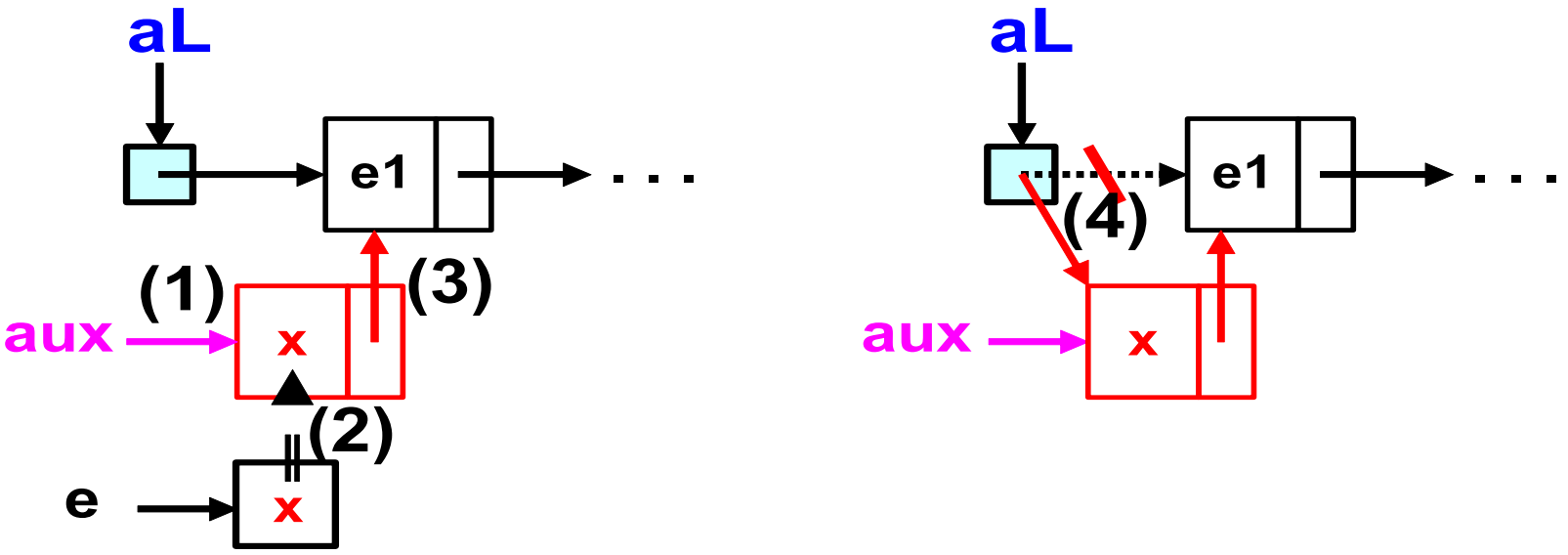

```
void AfiInv(TLista x)
{ if (!x) return;
  AfiInv (x->urm);
  printf ("%i  ", x->info);
}
```

Alocare Celula

```
TLista AlocCelula(TEL e)
/* adresa celulei create sau NULL */

{ TLista aux =
      (TLista)malloc(sizeof(TCelula)); /* (1) */
  if (!aux) return NULL;
  aux->info = e;  /* (2) */
  aux->urm = NULL; /* (3) */
  return aux;
}
```

Inserare la inceputul listei




```
Algoritm InsInc
{ incearca alocare celula; (1)
  daca esec atunci intoarce esec;
  completeaza info si urm din noua celula; (2, 3)
  actualizeaza inceput lista; (4)
  intoarce succes;
}
```

Inserare la inceputul listei

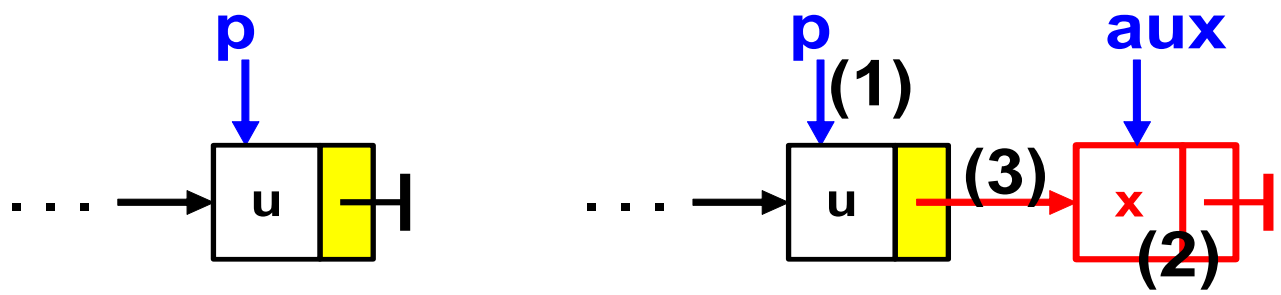
```
int InsInc(ALista aL, TEL e)
/* inserare la inceput reusita sau nu (1/0) */
{
TLista aux =
      (TLista)malloc(sizeof(TCelula)); /*(1)*/
if (!aux) return 0;
aux->info = e;  /* (2)*/
aux->urm = *aL;  /* (3)*/
*aL = aux;       /* (4) */
return 1;
}
```

Inserare la sfarsitul listei

Se realizeaza in doua etape:

A. Localizarea ultimei celule (in cazul particular de lista vida $\Rightarrow$ inserare la inceput)

B. Inserarea propriu-zisa



Functia va intoarce: adresa noii celule sau NULL, daca nu se poate aloca spatiu pentru aceasta;

Inserare la sfarsitul listei

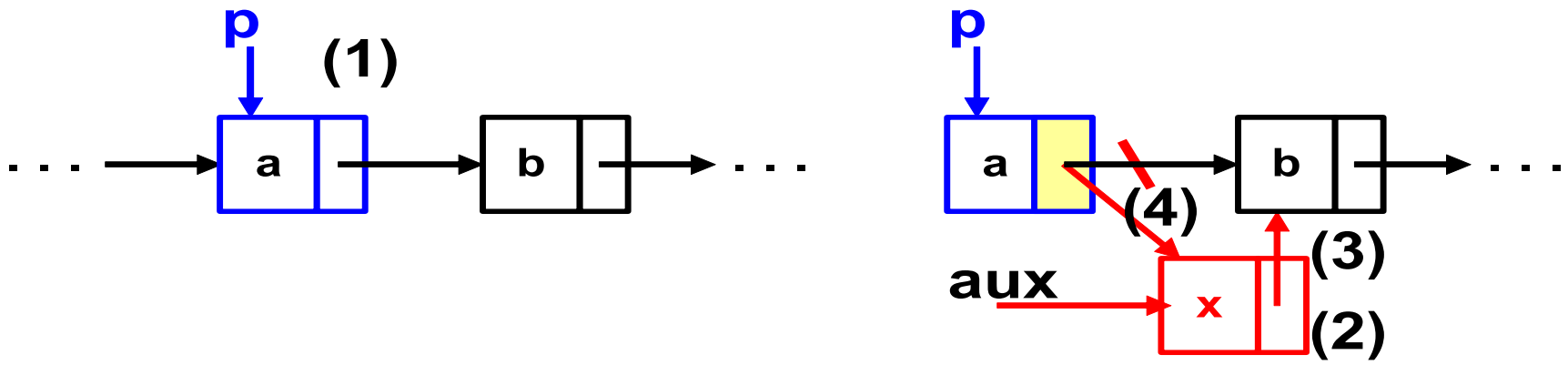
```
TLista InsSfL(ALista aL, TEL e)
/* adresa celulei inserate sau NULL */
{ TLista p = *aL, aux, ultim = NULL;
   while (p != NULL)
   {     ultim = p; p = p->urm; } /* (1) */
  aux= AlocCelula(e);  /* (2) */
  if(!aux) return NULL; /* intoarce NULL */
  if(ultim) ultim->urm = aux; /* (3) */
  else *aL = aux;
  return aux;
/* intoarce adresa celulei inserate */
}
```

Inserare dupa o anumita celula

Se realizeaza in doua etape:

A. Localizarea celulei dupa care se efectueaza inserarea (al carei camp urm se modifica) , folosind adresa celula (`p`) sau adresa legatura (`aL`)
Caz particular: celula inexistenta

B. Inserarea propriu-zisa.
Caz particular: alocare spatiu imposibila



Inserare dupa o anumita celula

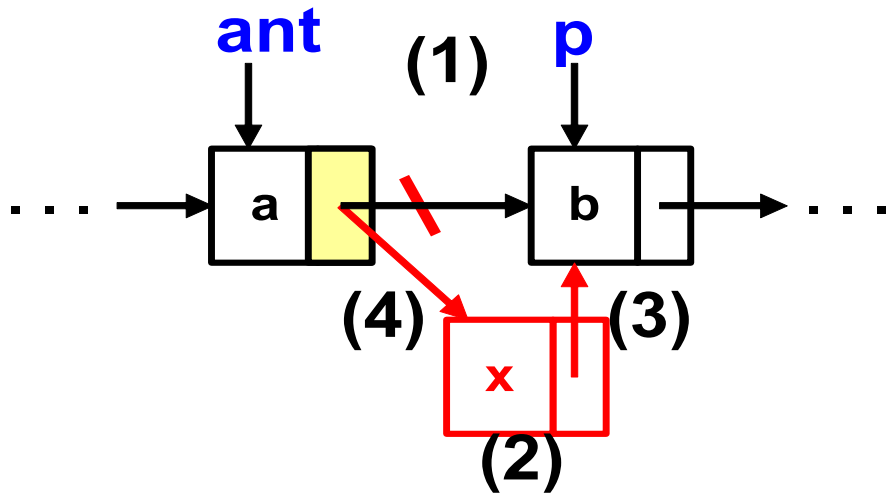
```
TLista InserareDupa(TLista L, TEL e, TEL ref)
/* intoarce adresa celula inserata sau NULL */
{ TLista aux, p;
  for(p = NULL; L != NULL; L = L->urm) /*(1) */
       if(L->info == ref) {p = L; break;}
  if (!p) return NULL;
  aux= AlocCelula(e);  /* (2) */
  if(!aux) return NULL;
  aux->urm = p->urm; /* (3) */
  p->urm = aux; /* (4) */
  return aux;
}
```

Inserare in fata unei celule

Se realizeaza in doua etape:

A. Localizarea legaturii modificate
(adresa listei sau adresa camp **`urm`**)
Caz particular: celula inexistenta

B. Inserarea propriu-zisa
Caz particular: alocare spatiu imposibila



Inserare in fata unei celule

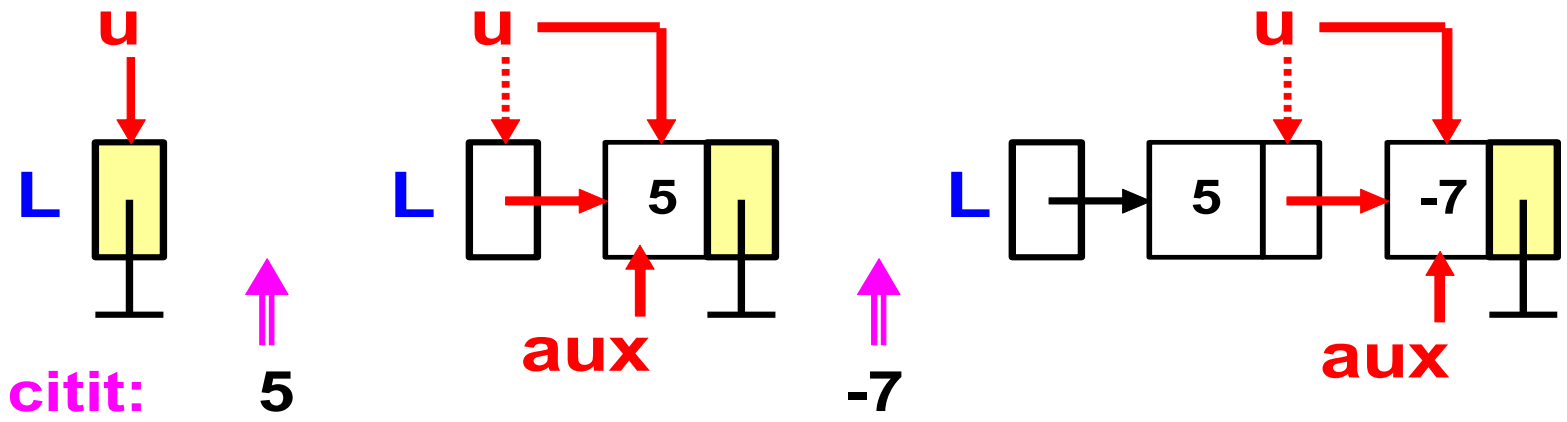
```
TLista InserareInainte
      (AListA aL, AEL ae, AEL aRef)
/* intoarce adresa celula inserata sau NULL */
{ TLista aux, ant, p, L;
  for(ant = NULL, p = NULL, L = *aL ;
     L != NULL; ant = L; L = L->urm) /* (1) */
     if(L->info == *aRef)
     {p = L; break;}

  if (!p) return NULL;
```

Inserare in fata unei celule

```
  aux= AlocCel(ae);  /* (2) */
  if(!aux) return NULL;
  if(ant == NULL)
  {
    aux->urm = p; /* (3) */
    *aL = aux;    /* (4) */
  }
  else
  {
    aux->urm = p; /* (3) */
    ant->urm = aux; /* (4) */
  }
  return aux;
}
```

Citire lista




```
{ initializeaza lista vida (L);
  cat timp s-a citit o valoare
  { insereaza urmatoarea celula;
    daca alocare esuata atunci iesire din bucla
  }
  intoarce L;
}
```

Citire lista (2)

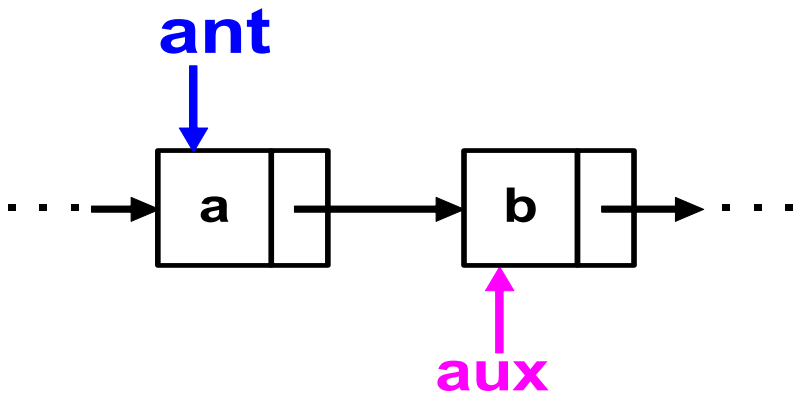
```
TLista CitireL(size_t*lg)
{ TLista L = NULL, u, aux;
  TInfo x;
  for(*lg=0; scanf("%i", &x) ==1;)
  { aux = AlocCel(&x);
    if(!aux) break;
    (*lg)++;
    if(L == NULL) L = aux;
    else u->urm = aux;
    u = aux;
  }
  return L;
}
```

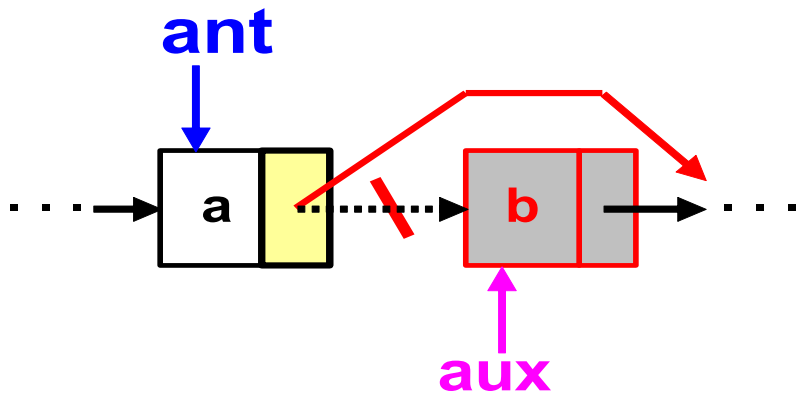
Eliminare element

Se realizeaza in doua etape:

A. Localizarea celulei dupa care se efectueaza eliminarea (al carei camp **urm** se modifica) .
Caz particular: celula inexistenta

B. Eliminarea propriu-zisa; daca este necesar, elementul din celula eliminata este copiat la o adresa furnizata ca parametru

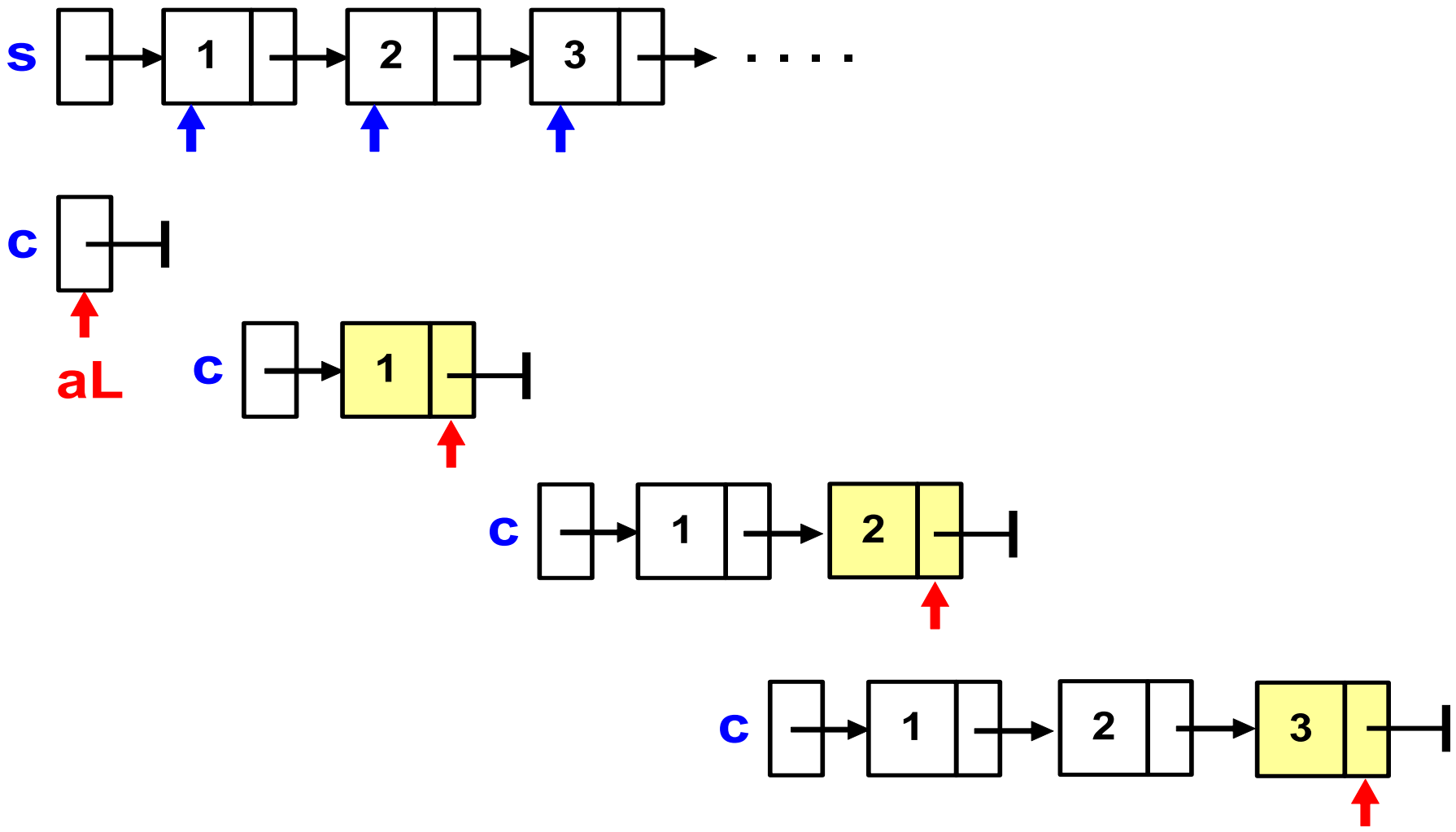




Distrugere lista

```
void DistrugeL(AListA aL)
{
  TLista aux;
  while(*aL)
  {
    aux = *aL;
    *aL = aux->urm;
    free(aux);
  }
  *aL = NULL;
}
```

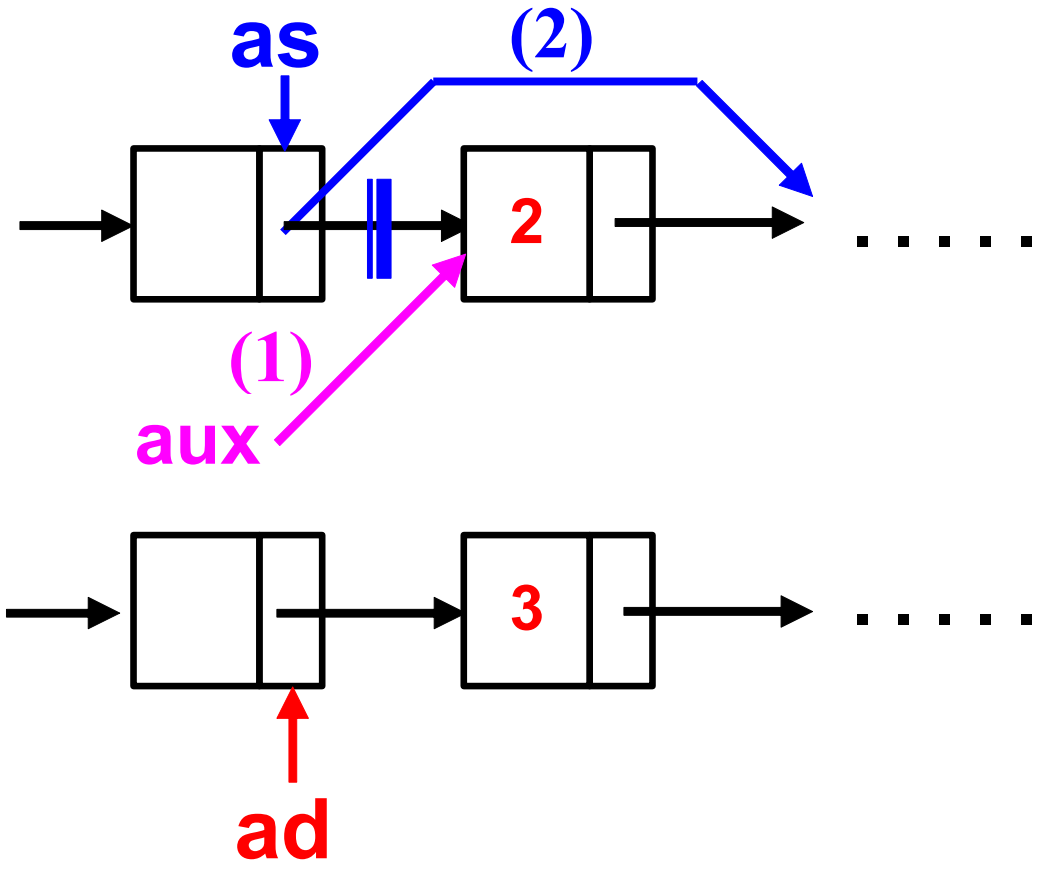
Copiere lista



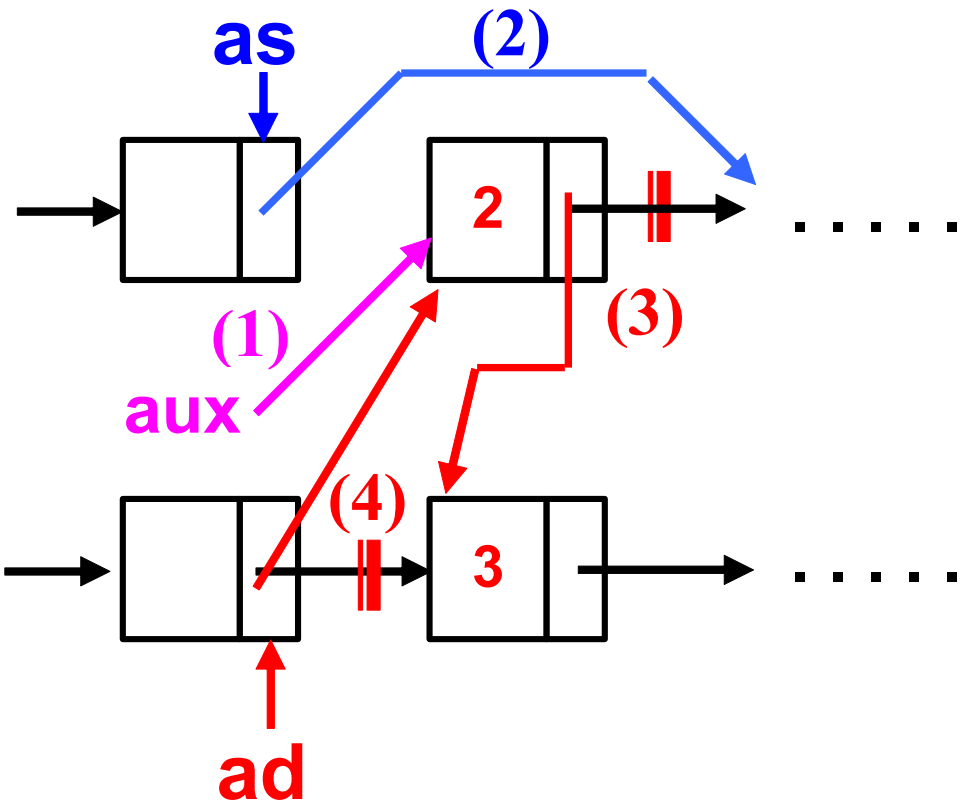
Copiere lista (2)

```
Algoritm Copiere
{ initializari;
  cat timp exista celula de copiat
  { daca nu reuseste alocare celula copie
      atunci
      { distruge copia partiala;
        intoarce NULL;
      }
      altfel
      { avans in copie partiala;
        avans in lista sursa;
      }
  }
  intoarce adresa copie;
}
```

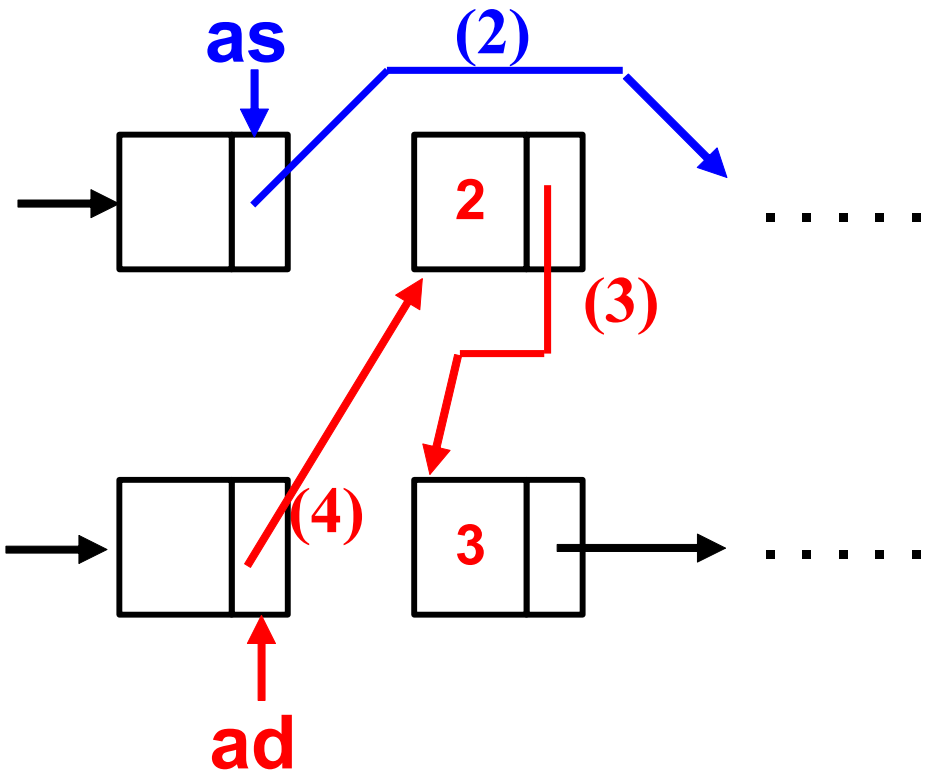
Mutare celula



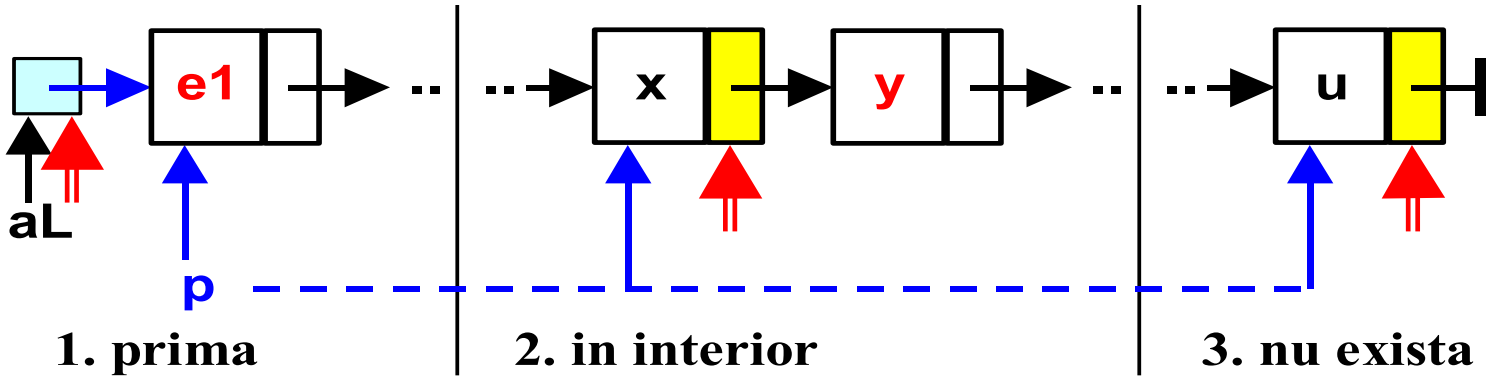
Mutare celula (2)

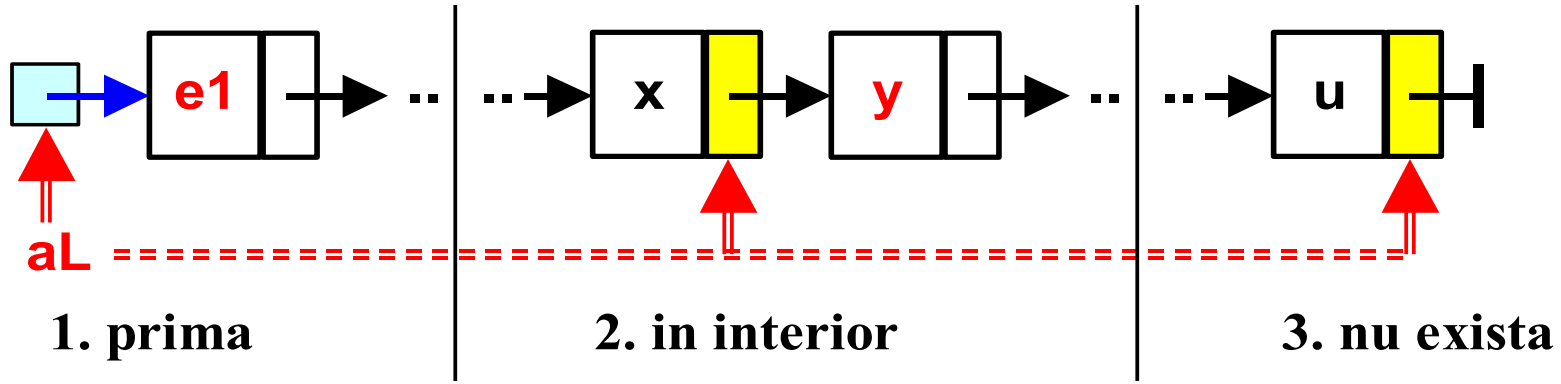


Mutare celula (3)

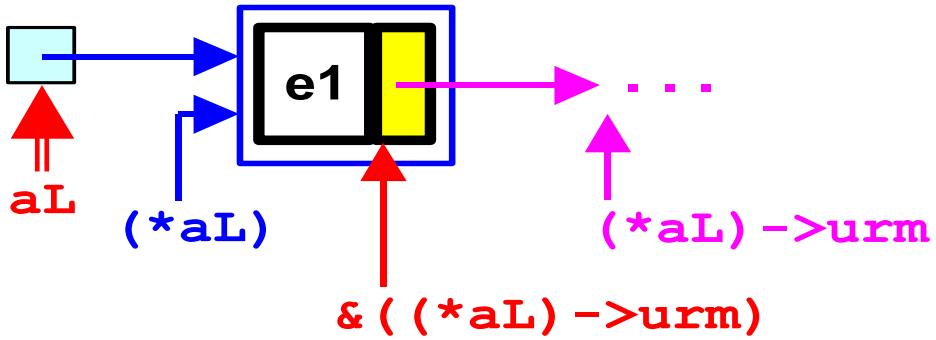


Cautare legatura spre celula





Cautare legatura spre celula



Daca se lucreaza numai cu adrese de legaturi, atunci avansul in lista se codifica astfel:

```
aL = &((*aL)->urm)
```

Cautare legatura spre celula

```
Alista Adr_Legatura(Alista aL, Tinfo* ae)
/* adresa legaturii catre prima aparitie
sau NULL */
{
   for(; *aL ; aL = &(*aL)->urm)
      if( (*aL)->info == *ae) reurn aL;
   return NULL;
}
```